

- The entire subtree rooted at node 12 moves up.
- The entire subtree rooted at 37 crosses over to become the left child of 50.
- The entire subtree rooted at 75 moves down as the right child of node 50.

The status messages indicate that the root node isn't black and the black heights differ. You can ignore these messages for the moment. Try rotating back and forth around the root by double-clicking the node and holding down the Shift key or using the second mouse button when rotating left. Do this (perhaps with the animation speed slowed down) and watch what happens to the subtrees, especially the one with 37 as its root.

In [Figure 10-22](#), the subtrees are enclosed in dotted triangles. Note that the rotation doesn't affect relations of the nodes within each subtree. The entire subtree moves as a unit. The subtrees can be larger (have more descendants) than the three nodes shown in this example. No matter how many nodes there are in a subtree, they will all move together during a rotation. Even when a subtree is empty, the movement is the same. The empty subtree goes to the same position it would have moved to if it had some nodes.

Inserting a New Node

Now you have enough background to see how a red-black tree's insertion routine could use rotations and the color rules to maintain the tree's balance. The remaining discussion describes the little details of exactly what conditions trigger what color changes and rotations to preserve balance.

Preview of the Insertion Process

Here, we briefly preview our approach to describing the insertion process. Don't worry if things aren't completely clear in the preview; we discuss this process in more detail in a moment.

The discussion that follows uses X, P, and G to designate a pattern of related nodes. X is a node that has caused a rule violation. Sometimes X refers to a newly inserted node, and sometimes to the child node when a parent and child have a red-red conflict.

- X is a particular node.
- P is the parent of X.
- G is the grandparent of X (the parent of P).

On the way down the tree to find the insertion point, you perform a color swap whenever you find a black node with two red children. Sometimes the swap causes a red-red conflict (a violation of Rule 3). Call the red child X and the red parent P. The conflict can be fixed with a single rotation or a double rotation, depending on whether X is an outside or inside grandchild of G. Following some color flips and rotations, you continue down to the insertion point and insert the new node.

After you've inserted the new node X, if P is black, you simply attach the new red node. If P is red, there are two possibilities: X is an outside or inside grandchild of G. You perform two color changes (we show what they are in a moment). If X is an outside grandchild, you perform one rotation, and if it's an inside grandchild, you perform two. This process restores the tree to a balanced state.

Now let's recapitulate this preview in more detail. We divide the discussion into three parts, arranged in order of complexity:

1. Color swaps on the way down
2. Rotations after the node is inserted
3. Rotations on the way down

If we were discussing these three parts in strict chronological order, we would examine part 3 before part 2. It's easier, however, to talk about rotations at the bottom of the tree than in the middle, and operations 1 and 2 are encountered more frequently than operation 3, so we discuss 2 before 3.

Color Swaps on the Way Down

The insertion routine in a red-black tree starts off doing essentially the same thing it does in an ordinary binary search tree: it follows a path from the root to the place where the node should be inserted, going left or right at each node, depending on the relative size of the node's key and the key to insert.

In a red-black tree, however, getting to the insertion point is complicated by color swaps and rotations. In the earlier “[Experiment 3: Color Swaps](#)” section, you learned the effects of changing colors; now let’s look at them in more detail.

Imagine the insertion routine proceeding down the tree, going left or right at each node, searching for the place to insert a new node. To make sure the color rules aren’t broken in the upcoming insertion, it needs to perform color swaps when necessary. Here’s the rule: every time the insertion routine encounters a black node that has two red children, it changes the children to black and the parent to red (unless the parent is the root, which always remains black).

How does a color swap affect the red-black rules? For convenience, let’s call the node at the top of the triangle, the one that’s red before the swap, P for parent. You can call P’s left and right children X1 and X2. This arrangement is shown on the left of [Figure 10-23](#).

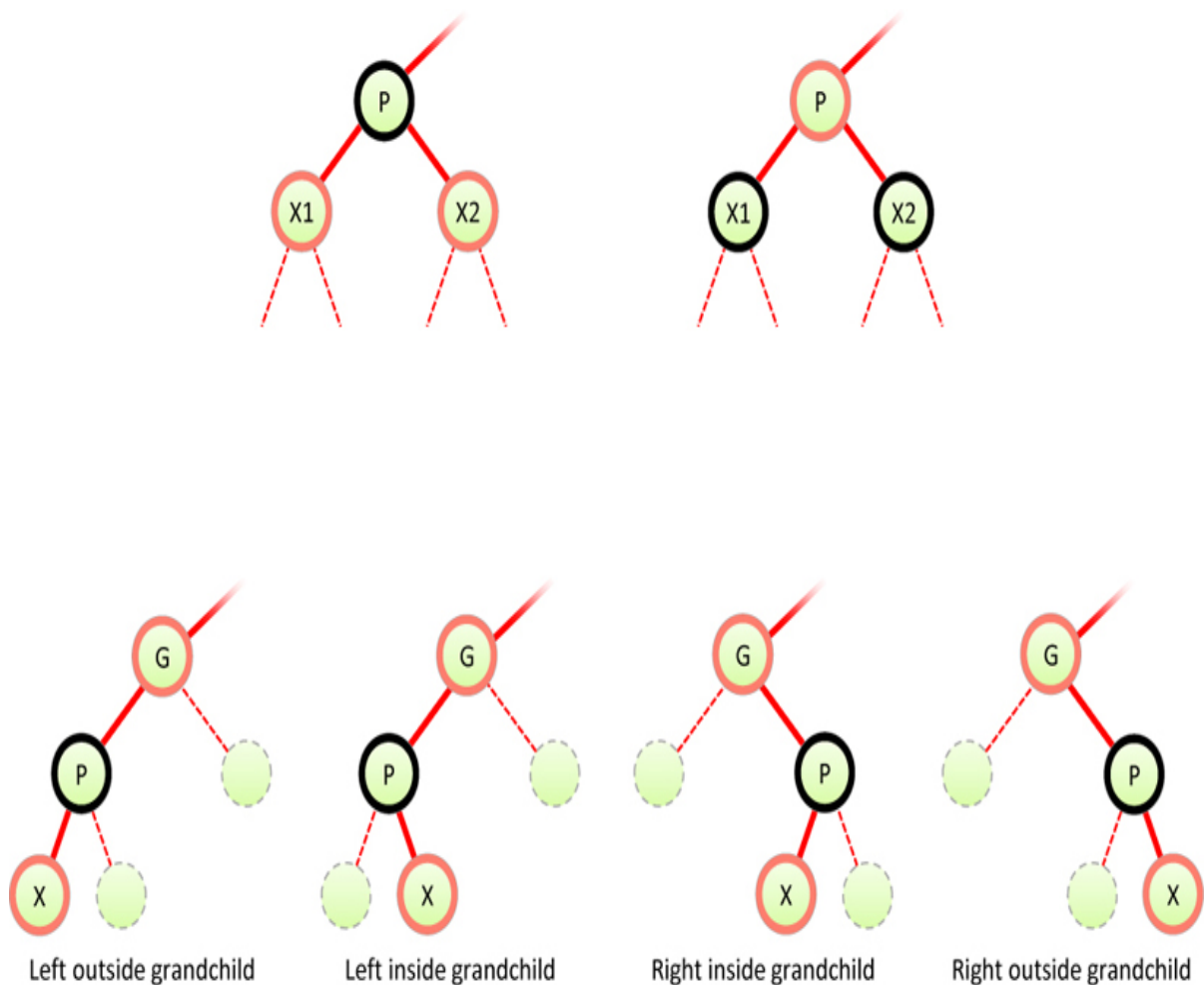


Figure 10-23 *Color swap*

Black Heights Unchanged

Figure 10-23 shows the nodes after the color swap on the right. The swap preserves the number of black nodes on the path from the root on down through P to any leaf or null node. All such paths go through P, and then through either X1 or X2. Before the swap (on the left of the figure), only P is black, so the triangle (consisting of P, X1, and X2) adds one black node to each of these paths.

After the swap, P is no longer black, but both X1 and X2 are, so again the triangle contributes one black node to every path that passes through it. In the exception where P is the root of the tree and the color swap leaves it black, one black node is added to every path in the tree. Thus, all the paths have the same black height, and a color swap can't cause Rule 4 to be violated.

Color swaps are helpful because they turn red leaf nodes into black leaf nodes. This design makes it easier to attach new red nodes without violating Rule 3.

Violation of Rule 3

Although Rule 4 is not violated by a color swap, Rule 3 (a node and its parent can't both be red) may be. If the parent of P is a black node, there's no problem when P is changed from black to red. If the parent of P is red, however, the color change creates two linked red nodes.

This problem must be fixed before you continue down the path to insert the new node. You can correct the situation with a rotation, as we'll soon see.

Inserting at a Leaf

After you've worked your way down to the appropriate place in the tree, performing color swaps (and rotations) if necessary, on the way down, you can then insert the new node as described in [Chapter 8](#) for an ordinary binary search tree. That's not, however, the end of the story.

Rotations After the Node Is Inserted

The insertion of the new node may cause the red-black rules to be violated. Figure 10-20 showed the example of inserting a node with key 6 that violates

Rule 3. Therefore, following the insertion, you must check for rule violations and take appropriate steps.

Remember that, as described earlier, the newly inserted node, which is called X, is always red. X may be located in exactly four positions relative to P and G, as shown in [Figure 10-24](#).



Figure 10-24 *Handed variations of node being inserted*

Remember that a node X is an outside grandchild if it's on the same side of its parent, P, that P is of its parent, G. That is, X is an outside grandchild if either it is a left child of P and P is a left child of G, or it is a right child of P and P is a right child of G (as in the two outermost relations of [Figure 10-24](#)).

Conversely, X is an inside grandchild if it's on the opposite side of its parent, P, that P is of its parent, G (as in the inner relations of [Figure 10-24](#)).

The multiplicity of what might be called “handed” (left or right) relationships between the inserted node and its ancestors is one reason the red-black insertion routine is so complex to program.

The action needed to conform to the red-black rules is determined by the colors and configuration of X and its relatives. Perhaps surprisingly, nodes can be arranged in only three major ways (not counting the handed variations already mentioned). Each possibility must be dealt with in a different way to preserve red-black correctness and thereby lead to a balanced tree. We list the three possibilities briefly and then discuss each one in detail in its own section. [Figure 10-25](#) shows what they look like. Remember that X is always red.

1. P is black.

2. P is red and X is an outside grandchild of G.
3. P is red and X is an inside grandchild of G.

You might think that this list doesn't cover all the possibilities. We return to this question after we explored these three.

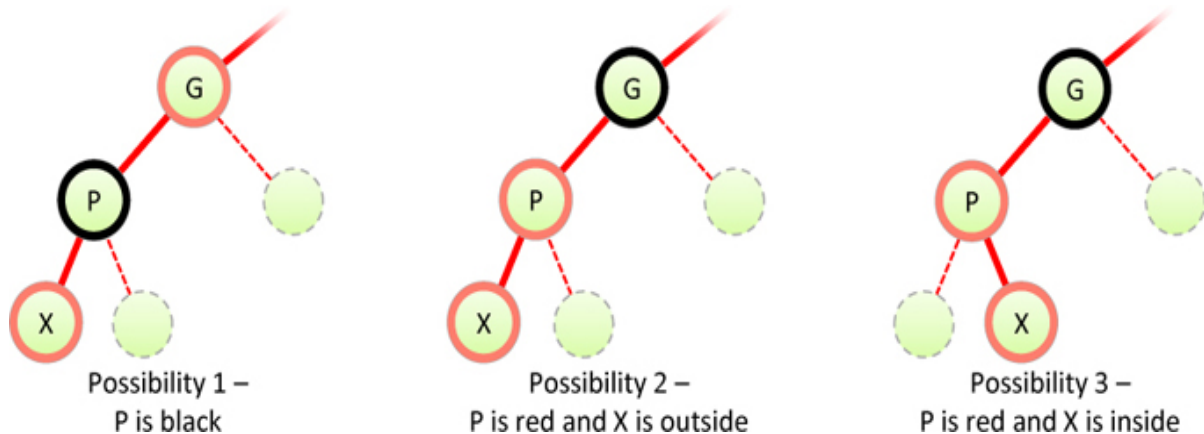


Figure 10-25 *Three post-insertion possibilities*

Possibility 1: P Is Black

If P is black, you get a free ride. The node you've just inserted is always red. If its parent is black, there's no red-to-red conflict (Rule 3) and no addition to the number of black nodes (Rule 4). Thus, no color rules are violated. You don't need to do anything else. The insertion is complete.

Possibility 2: P Is Red and X Is Outside

If P is red and X is an outside grandchild, you need a single rotation and some color changes. The single rotation is the same as what was needed in an AVL tree, but the color changes are new. Let's return to the example in [Figure 10-20](#), which was set up in the RedBlackTree Visualization tool by inserting 50, 20, 71, 12, and 6 in an empty tree. [Figure 10-26](#) shows the situation along with the X, P, and G labels. The status in the Visualization tool says there is 1 red-red link, so you know you need to take some action.



Figure 10-26 *P is red and X is an outside grandchild*

In this situation, you can take three steps to restore red-black correctness and thereby balance the tree. Here are the steps:

1. Switch the color of X's grandparent G (20 in this example).
2. Switch the color of X's parent, P (12).
3. Rotate with X's grandparent, G (20), at the top, in the direction that raises X (6). This is a right rotation in the example.

As you've learned, to flip colors in the Visualization tool, single-click the node with a pointer device. To rotate right, double-click the top node. (Alternatively, single click the parent, 12, then single click the grandparent, 20, and then select the Rotate Right button while 20 has already been filled in as the rotation point). After you've completed the three steps, the Visualization tool will inform you that the tree is red-black correct. It's also balanced, as shown at the right of [Figure 10-26](#).

In this example, X was an outside grandchild and a left child. There's a symmetrical situation when the X is an outside grandchild but a right child. Try this by creating the tree 50, 25, 75, 87, 93. Fix it by changing the colors of 87 and 75, and rotating left at node 75. Again, the tree becomes balanced.

Possibility 3: P Is Red and X Is Inside

If P is red and X is an inside grandchild, you need two rotations and some color changes. To see this one in action, use the Visualization tool to create the tree with keys 50, 20, 71, 12, 17. The result is the tree on the left of [Figure 10-27](#).



Figure 10-27 *P is red and X is an inside grandchild*

Note that node 17 is an inside grandchild. Both it and its parent are red, so again you see the status message that there is one red-red link.

Fixing this arrangement is slightly more complicated. If you try to rotate right with the grandparent node, G (20), at the top, as you did in Possibility 2, the inside grandchild, X (17), moves across rather than up, so the tree is no more balanced than before. (Try this; then rotate back left, with 12 at the top, to restore it.) A different solution is needed.

The trick when X is an inside grandchild is to perform *two* rotations rather than one, just as was needed for AVL trees. Rotation 1 changes X from an inside grandchild to an outside grandchild, as shown in middle of [Figure 10-27](#). Now the situation is like Possibility 1, and you can apply the same rotation, with the grandparent at the top, as you did before. The result is shown on the right.

You must also recolor the nodes. You do this before doing any rotations. The order of these operations doesn't really matter, but if you wait until after the rotations to recolor the nodes, it's hard to know what to call them. Here are the steps:

1. Switch the color of X's grandparent (20 in this example).
2. Switch the color of X (*not* its parent; X is 17 here).
3. Rotate with X's parent, P, at the top (*not* the grandparent; the parent is 12), in the direction that raises X (a left rotation in this example).
4. Rotate again with X's grandparent, G (20), at the top, in the direction that raises X (a right rotation).

The rotations and recoloring restore the tree to red-black correctness and balance it. As with Possibility 2, there is a mirror image case in which P is the right child of G rather than the left.

What About Other Possibilities?

Do the three Post-Insertion Possibilities just discussed really cover all situations?

Suppose, for example, that X has a sibling, S, the other child of P. This scenario might complicate the rotations necessary to insert X and would be as if the bottom dashed nodes in each of the possibilities of [Figure 10-25](#) were filled in. If P is black, then there's still no problem inserting X, which is red (that's Possibility 1).

What about when P is red? Well, you know that the tree was balanced just before the insertion, so G must be black when P is red (to avoid violating Rule 3). If there were a sibling of X, it too would have to be black (to avoid violating Rule 3). That would mean, just before the insertion, P had only one child, the sibling, and it was black. That's not balanced because P's null child would have a different number of black nodes in its path, violating Rule 4. You can conclude that it's impossible for X to have a sibling when P is red.

Another case to explore is that G, the grandparent of P, has a child, U, the sibling of P and the uncle of X (the top dashed nodes in [Figure 10-25](#)). Again, this scenario might complicate any necessary rotations. If P were black, there would be no need for rotations when inserting X, as you've seen. Let's assume P is red and G is black (as in Possibilities 2 and 3 of [Figure 10-25](#)). Then U must also be red; otherwise, the black height going from G to P's null child would be different from that going from G to U. Because a black parent, G, with two red children, P and U, would be swapped on the way down, you conclude that this situation can't exist either.

Thus, the three possibilities just discussed are the only ones that can exist (except for the mirror cases of Possibilities 2 and 3, X can be a right or left child and P can be a right or left child).

What the Color Swaps Accomplished

Suppose that performing a rotation and appropriate color changes caused other violations of the red-black rules to appear further up the tree. You can imagine

situations in which you would need to work all the way back up the tree, performing rotations and color switches, to remove rule violations. That's what was done in AVL trees.

Fortunately, this situation can't arise. Using color swaps on the way down eliminates the situations in which a rotation could introduce any rule violations further up the tree (we cover rotations on the way down in the next section). When you get to inserting at the leaf, it's guaranteed that one or two rotations plus two color flips will restore red-black correctness in the entire tree. Proving this guarantee is beyond the scope of this book, but such a proof is possible.

The color swaps on the way down make insertion in red-black trees more efficient than in other kinds of balanced trees, such as AVL trees. They ensure that you need to pass through the tree only once, on the way down. If the implementation is recursive, no work is needed after the recursive call returns.

Rotations on the Way Down

Now let's discuss the last of the three operations involved in inserting a node: making rotations on the way down to the insertion point. As we noted, although we discuss this operation last, it actually takes place before the node is inserted. We waited until now to discuss it only because it was easier to explain rotations for a just-installed node than for nodes in the middle of the tree.

In the discussion of color swaps during the insertion process, we noted that a color swap could cause a violation of Rule 3 (a parent and child can't both be red). The problem arises when you visit a black node with two red children and a red parent. We also noted that a rotation could fix this violation.

On the way down there are two possibilities, corresponding to Possibility 2 and Possibility 3 during the insertion phase described earlier. The offending node can be an outside grandchild, or it can be an inside grandchild. (In the situation corresponding to Possibility 1, no action is required.)

Outside Grandchild

First, let's examine an example in which the offending node is an outside grandchild. By "offending node," we mean the child in the parent-child pair that caused the red-red conflict.

For this example, start a new tree and insert the following keys: 50, 25, 75, 12, 37, 6, and 18. The insertion of 12 and 6 will cause color swaps.

Now try to insert a node with the value 3. The tree initially looks like the one on the left in [Figure 10-28](#). As the search for the insertion point unfolds, it recognizes the need to swap colors at the subtree rooted at node 12. When the colors are swapped, node 12 becomes red, matching its parent, node 25. The visualization tool proceeds to insert node 3, but let's look at the situation right after the color swap at node 12.

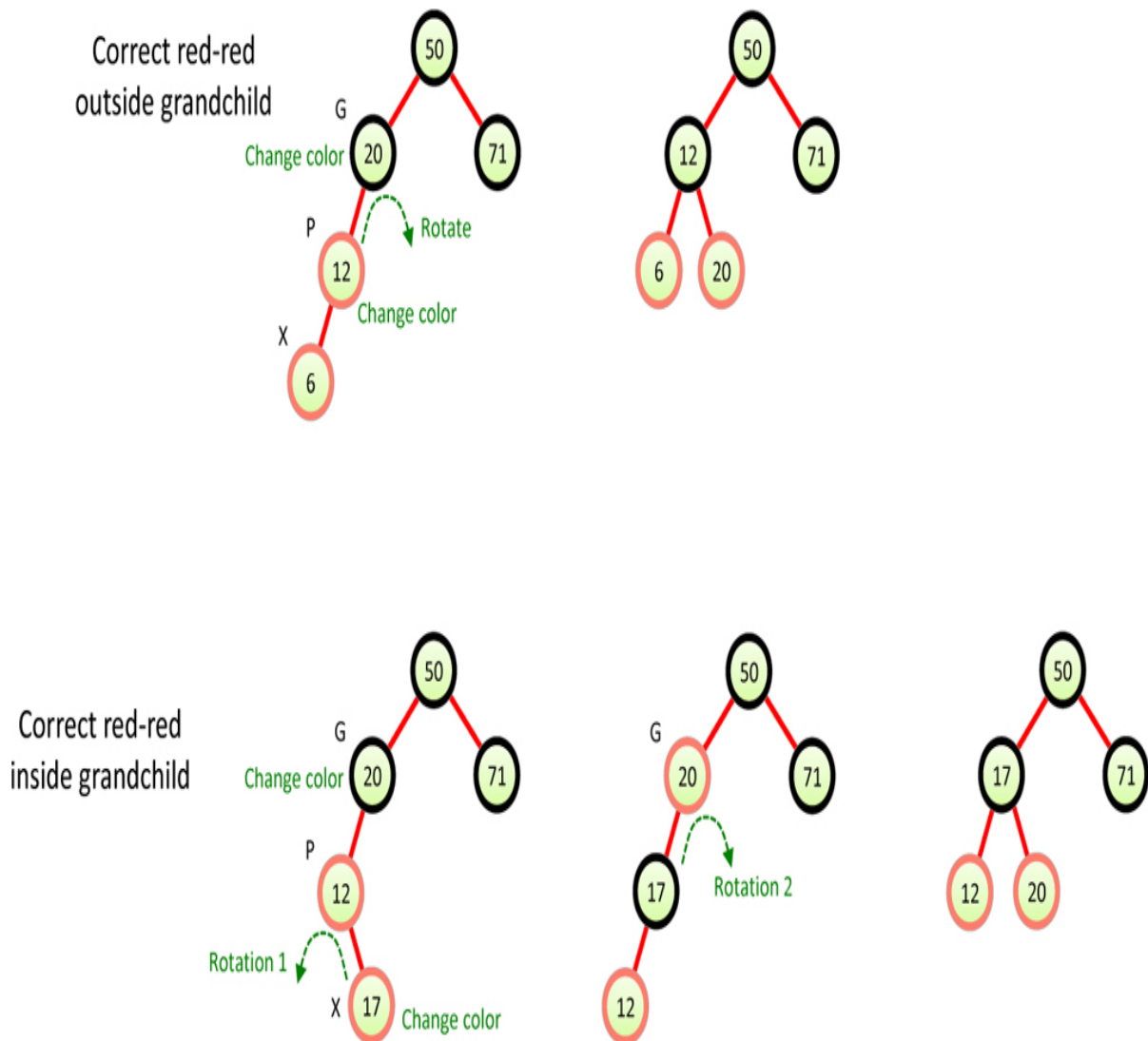


Figure 10-28 *Correcting a red-red outside grandchild on the way down*

The middle tree in [Figure 10-28](#) shows the red-red conflict where node 12 is the problem, X; node 25 is its parent, P; and node 50 is its grandparent, G.

Similar to the way you handle insertion at the leaf level, you must perform two color changes and one rotation. This example looks a little odd because X referred to the node being inserted in the earlier discussion, and here it's not even a leaf node. These on-the-way-down rotations, however, can take place anywhere within the tree.

You follow the same set of rules you did under Possibility 2: P Is Red and X Is Outside, discussed earlier:

1. Flip the color of X's grandparent G (50 in this example). Ignore the status message about the root not being black.
2. Flip the color of X's parent P (25).
3. Rotate with X's grandparent (50) at the top, in the direction that raises X (here a right rotation).

These steps result in the tree on the right of [Figure 10-28](#). Suddenly, the tree is not only balanced but has also become pleasantly symmetrical! This outcome appears to be a bit of a miracle, but it's only the result of following the color rules.

Now the node with value 3 can be inserted in the usual way. Because the node it connects to, 6, is black, there's no complexity about the insertion.

When you try this in the visualization tool, it doesn't perform the rotation automatically during the insertion of node 3; you will have to do it after the insertion. During the animation, when the `current` arrow reaches node 12, the tool shows the top panel in [Figure 10-29](#).

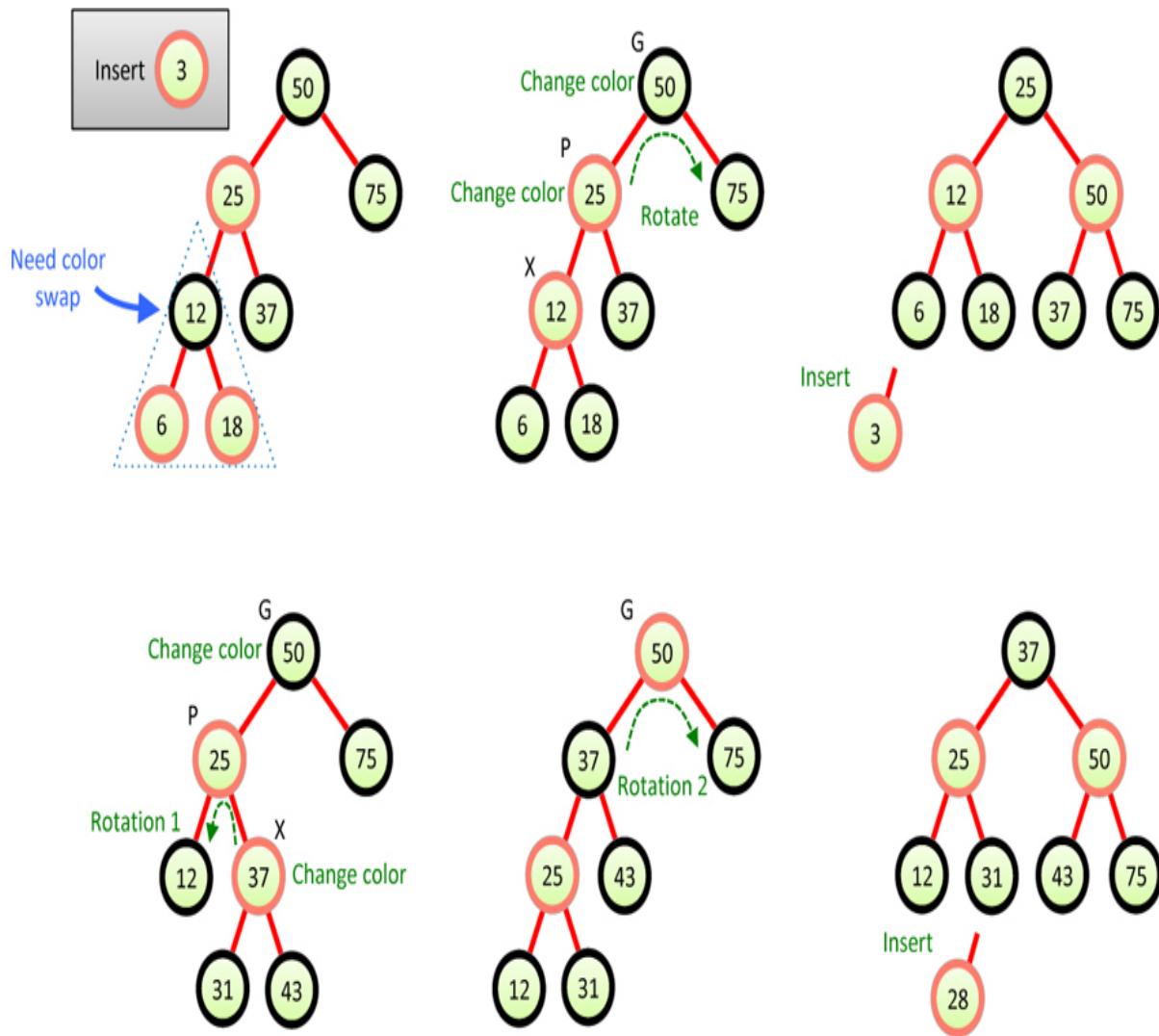


Figure 10-29 *Correcting a red-red outside grandchild in the Visualization tool*

The same color flips and rotation can be performed after the insertion to satisfy the red-black rules. In fact, if the algorithm were omniscient, it could perform the color flips and rotation on the original tree, even before an insertion was attempted. Sadly, omniscience is not an option. It's quite practical, however, to detect and fix the problem on the downward search.

Inside Grandchild

If X is an inside grandchild when a red-red conflict occurs on the way down, two rotations are required to set it right. This situation is like that of the inside grandchild in the post-insertion phase, which we called Possibility 3.

To see it in action, empty the tree in the visualization tool, and insert 50, 25, 75, 12, 37, 31, and 43. Now try to insert a new node with the value 28. During the animation of the `current` arrow descending, it swaps node 37's color with that of its children. That leaves node 37 red and in conflict with its parent, node 25, as shown at the left of [Figure 10-30](#). In this situation G is 50, P is 25, and X is 37.



Figure 10-30 *Correcting a red-red inside grandchild on the way down*

To cure the red-red conflict, you must do the same two color flips and two rotations as described earlier in “[Possibility 3: P Is Red and X Is Inside](#)” :

1. Change the color of G (node 50; ignore the message that the root is not black).
2. Change the color of X (37).
3. Rotate with P (25) as the top, in the direction that raises X (left in this example). That produces the tree in the middle of [Figure 10-30](#).
4. Rotate with G (50) as the top, in the direction that raises X (right in this example).

Now the insertion of node 28 can proceed as shown at the right of [Figure 10-30](#). Again, this operation poses no additional challenges because both nodes 31 and 43 were swapped to black when you first tried to insert 28.

When you use the visualization tool to insert 28, it will perform the color swap of nodes 37, 31, and 43. You will have to perform the preceding four steps after the insertion of 28 is done.

We've concluded the description of how a tree is kept red-black correct during the insertion process. Adherence to the red-black rules guarantees balance because the level of leaves can only differ by at most two within any subtree (forgetting about the red and black labels). That's a different definition of balanced when compared to AVL trees, and we look at what that means when we discuss efficiency.

Deletion

As you may recall from [Chapter 8](#) and saw in the implementation of AVL trees earlier, coding for deletion is harder than for insertion. [Chapter 9](#) described how deletion could be done in 2-3-4 trees using rotation and fusion operations to maintain balance. With red-black trees, rotation and color change operations can be done to maintain red-black correctness (balance), and the deletion process remains, as you might expect, quite complex.

In fact, the deletion process is so complicated that many programmers sidestep it in various ways. One approach, as with ordinary binary trees, is to mark a node as deleted without actually deleting it—a “soft” delete. Any search routine that finds the node knows not to report it to the caller. Deleted nodes must still maintain their red-black status, however, in order to balance the overall tree. The soft delete solution works in situations where deletions are not a common occurrence. That approach, however, has a way of ending up causing performance issues when someone takes the implementation and applies it to some problem where deletions happen more frequently. In any case, we forgo a discussion of the deletion process.

The Efficiency of Red-Black Trees

Like ordinary binary search trees, a red-black tree allows for searching, insertion, and deletion in $O(\log_2 N)$ time, which is the same as $O(\log N)$ time. Search times should be almost the same in the red-black tree as in the ordinary tree because the red-black characteristics of the tree aren't used during searches. The difference comes, however, in the height of the red-black trees. The rule that constrains the number of levels, Rule 4, only counts the black nodes on the path. That means, that there can potentially be a path to a black leaf that has exactly two black nodes in it, while a path to a red leaf has two black nodes and two red nodes, as in [Figure 10-31](#). The count of black nodes is

the same, but the path followed by a search is twice as long. According to Sedgwick (see [Appendix B](#)), it can be shown that the search cannot require more than $2 \times \log_2 N$ comparisons. That constant 2 in front comes from the extra path length, and it's the worst case. On average it takes about $\log_2 N$ comparisons. In big O notation, remember, you ignore the constant so it remains $O(\log N)$.

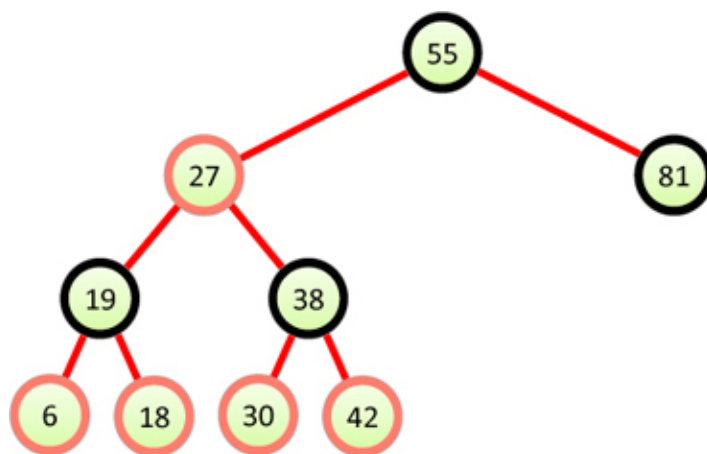


Figure 10-31 *A red-black tree whose longest path is twice the shortest path*

The times for insertion and deletion are increased by a constant factor because of having to perform color swaps and rotations on the way down and at the insertion point. On average, an insertion requires about one rotation. Therefore, insertion still takes $O(\log N)$ time but is slower than insertion in the ordinary binary tree. Compared with the AVL tree, which performs rotations bottom-up, the red-black tree is about twice as fast because it only needs to do work during the descent, although the other multipliers for color flips and rotations, plus the possibility of being deeper, change that a bit.

The storage penalty compared to ordinary binary search trees is the Boolean variable needed to label the color of each node. That's smaller than what the AVL tree required. It's also independent of the number of levels in the tree (you don't need more space per node when the number of levels gets huge).

Because in most applications there will be more searches than insertions and deletions, there is probably not much overall time penalty for using a red-black tree or an AVL tree instead of an ordinary binary search tree. Of course, the big advantage of red-black, AVL, and 2-3-4 trees is their insensitivity to the order

items are inserted. None of them slow down to $O(N)$ search performance, even if the data is inserted in the order of the keys.

2-3-4 Trees and Red-Black Trees

At this point 2-3-4 trees (described in [Chapter 9](#)) and red-black trees probably seem like entirely different entities. It turns out that in a certain sense they are completely equivalent. One can be transformed into the other by the application of a few simple rules, and even the operations needed to keep them balanced are equivalent. Mathematicians would say they are **isomorphic**.

You probably won't ever need to transform a 2-3-4 tree into a red-black tree, but the equivalence of these structures casts additional light on their operation and is useful in analyzing their efficiency.

Historically, the 2-3-4 tree was developed first; later the red-black tree evolved from it.

Transformation from 2-3-4 to Red-Black

As you might recall, the nodes of a 2-3-4 tree have either 2, 3, or 4 child links (possibly all empty for a leaf node). Each 2-3-4 node can be transformed into a red-black subtree by applying a rule based on its type, as shown in [Figure 10-32](#). The rules are

- Transform any 2-node in the 2-3-4 tree into a one black node in the red-black tree. Its two children, W and X, become the left and right children of the black node.
- Transform any 3-node into a child node, C, and a parent node, P. There are two possibilities for where to transform the three original child links—W, X, and Y—as shown in [Figure 10-32](#). The child node C gets either W and X or X and Y as its child links. The parent, P, gets the one remaining child: either Y or W. It doesn't matter which item becomes the child and which the parent if the relationships between the keys are preserved. The parent is colored black, and the child is colored red.
- Transform any 4-node into a parent and two children. The first child gets children W and X; the second child gets children Y and Z. As before, the parent is black; the two children are colored red.

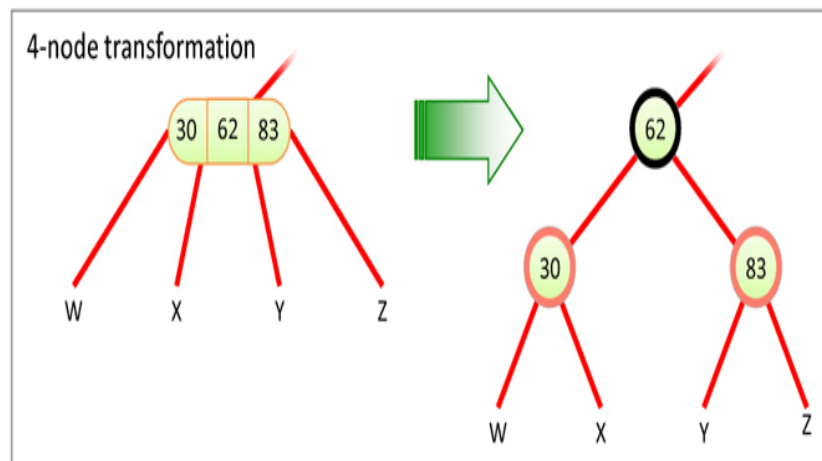
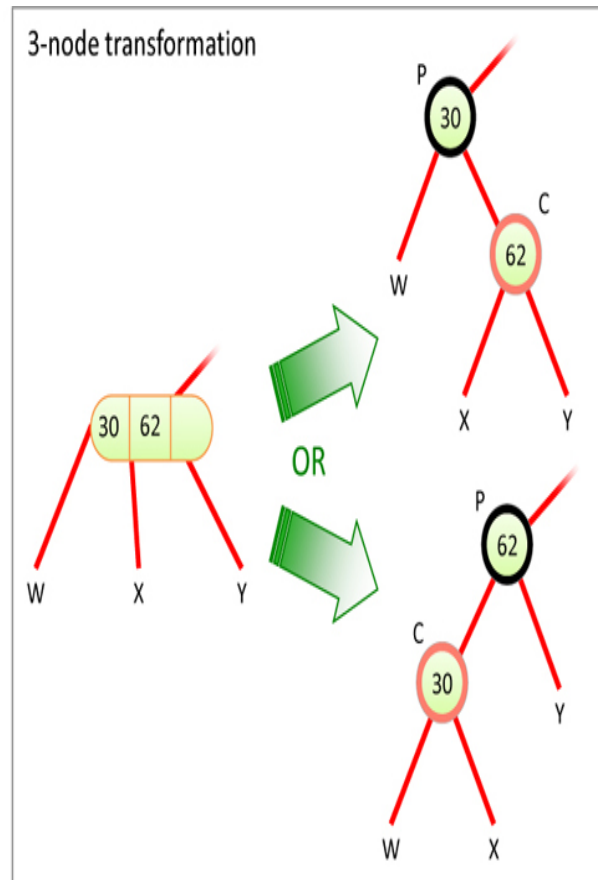
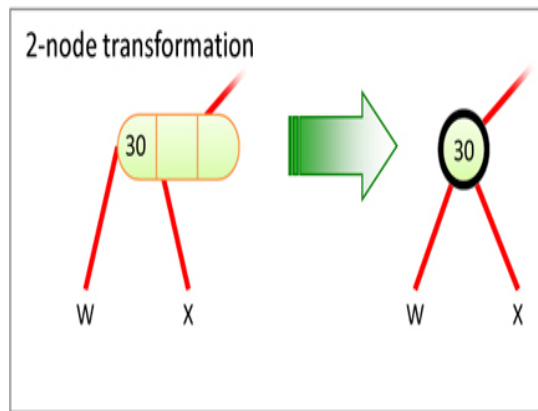


Figure 10-32 *Transforming nodes: 2-3-4 to red-black*

Note that a leaf node storing a single item is considered a 2-node when applying these rules. In general, a leaf node storing L items transforms as a $L+1$ node does.

Figure 10-33 shows a 2-3-4 tree and its corresponding red-black tree obtained by applying these transformations. Dotted triangles surround the subtrees that

were made from 4-nodes (one black and two red nodes). Dotted lozenges surround the subtrees that were made from 3-nodes (one black and one red node). The red-black rules are automatically satisfied by the transformation. Check that this is so: the root is black, two red nodes are never connected, and for every leaf and null child, the path to the root has the same number of black nodes, 3.

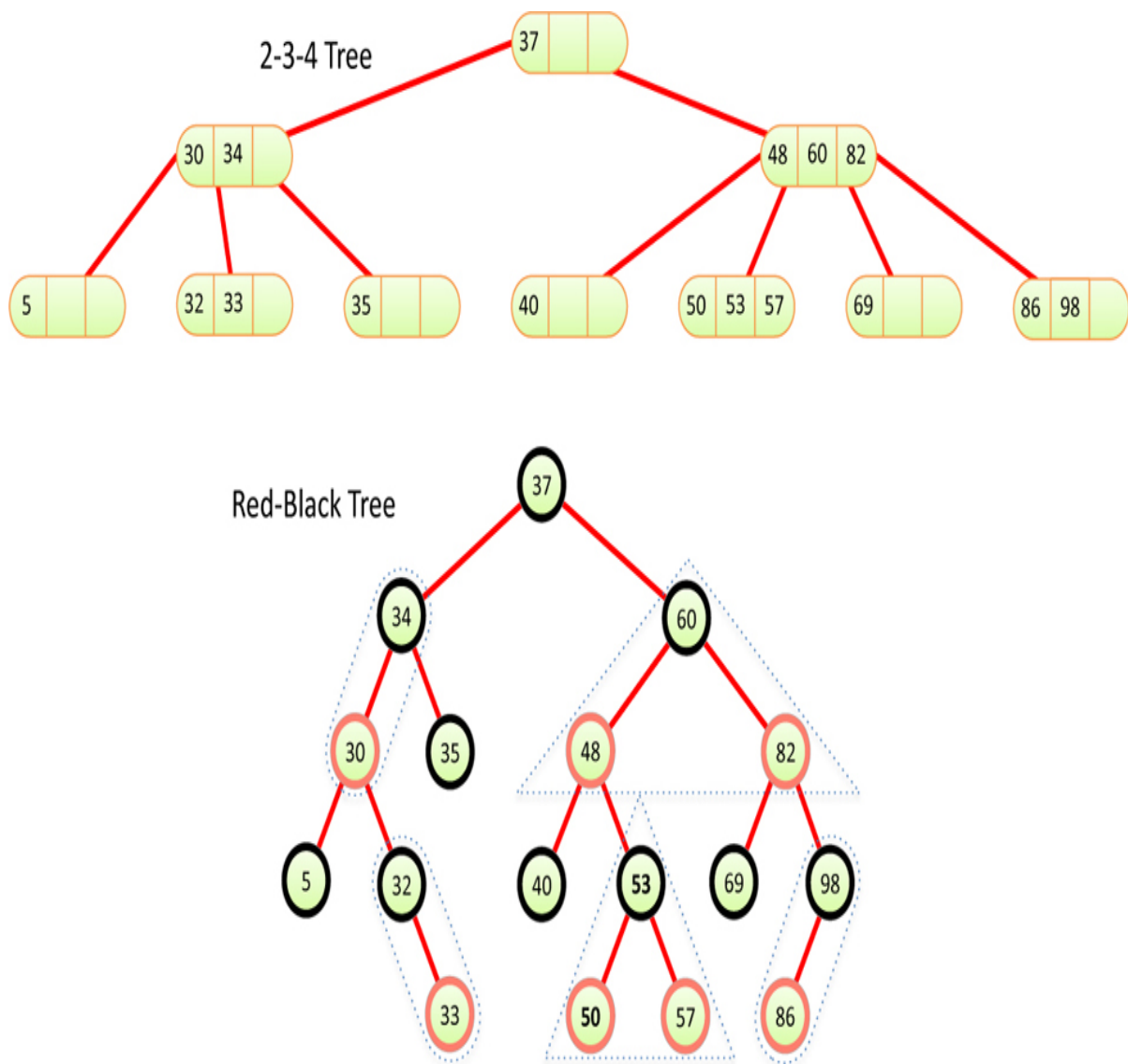


Figure 10-33 *A 2-3-4 tree and its red-black equivalent*

You can say that a 3-node in a 2-3-4 tree is equivalent to a parent with a red child in a red-black tree, and a 4-node is equivalent to a parent with two red children. It follows that a black parent with a black child in a red-black tree does *not* represent a 3-node in a 2-3-4 tree; it simply represents a 2-node with

another 2-node child. Similarly, a black parent with two black children (like the root node of the red-black tree in [Figure 10-33](#)) does not represent a 4-node.

Operational Equivalence

Not only does the structure of a red-black tree correspond to a 2-3-4 tree, but the operations applied to these two kinds of trees are also equivalent. In a 2-3-4 tree the tree is kept balanced during insertion using node splits. In a red-black tree the balancing methods are color swaps, flips, and rotations.

4-Node Splits and Color Swaps

As the insertion algorithm descends a 2-3-4 tree searching for where a new node goes, it splits each 4-node into two 2-nodes. In a red-black tree the algorithm calls for color swaps. Are these operations equivalent?

The top row of [Figure 10-34](#) shows a 4-node (with keys 40-50-60) in a 2-3-4 tree being split. The 2-node holding item 70 that was the parent of the 4-node becomes a 3-node, holding 50 and 70.

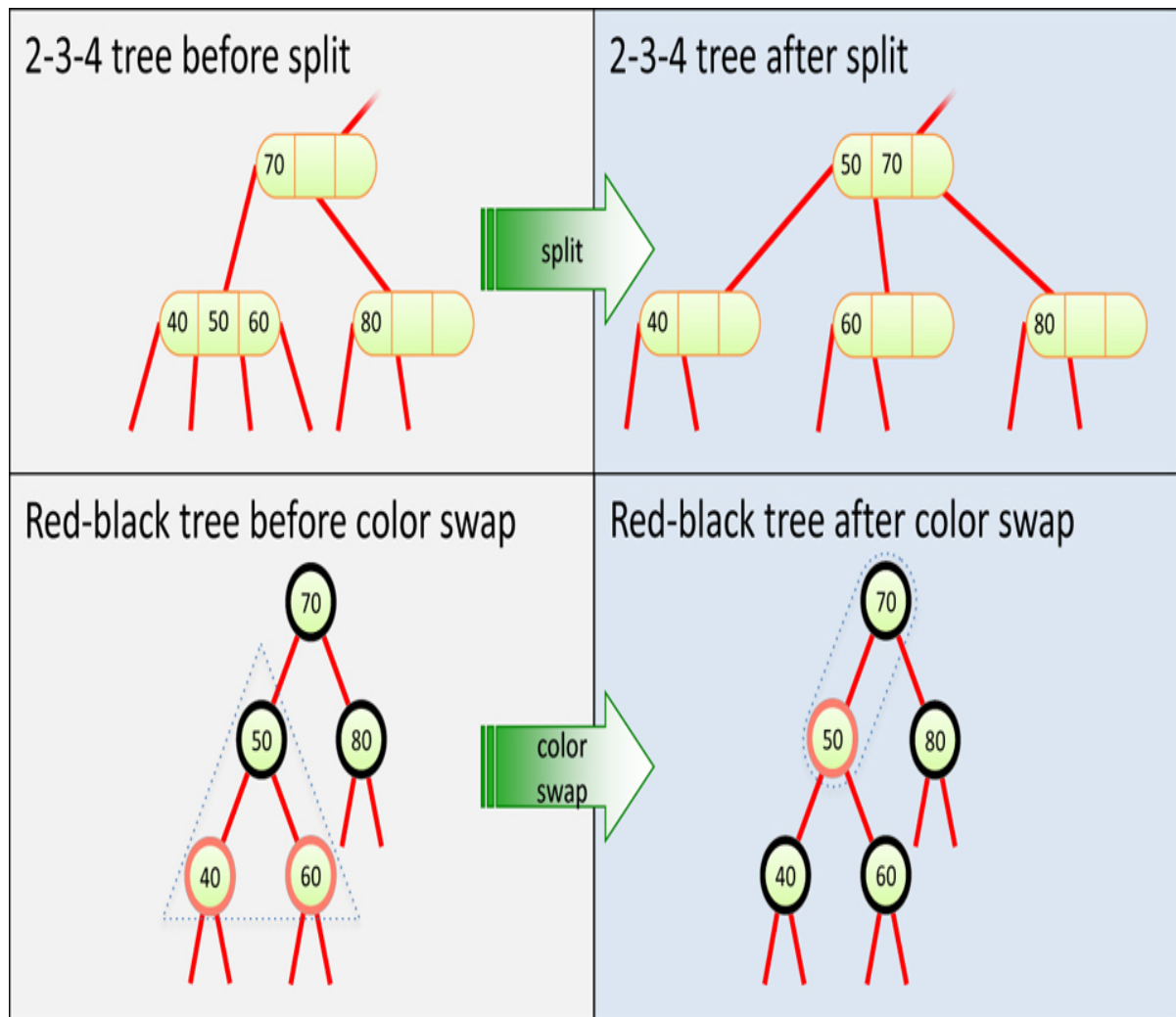


Figure 10-34 A 4-node split and color swap

The bottom row of [Figure 10-34](#) shows the red-black equivalent to the 2-3-4 tree in the top row. The dotted triangle surrounds the equivalent of the 4-node, holding 40, 50, and 60. A color swap causes nodes 40 and 60 to become black and node 50 to become red. Thus, node 50 and its parent form the equivalent of a 3-node, as shown by the dotted lozenge shape surrounding nodes 50 and 70. This is equivalent to the 3-node holding 50 and 70 above.

Thus, it's clear that splitting a 4-node during the insertion process in a 2-3-4 tree is equivalent to performing color swaps during the insertion process in a red-black tree.

3-Node Splits and Rotations

When a 3-node in a 2-3-4 tree is transformed into its red-black equivalent, two arrangements are possible, as shown earlier in [Figure 10-32](#). Either of the two data items can be placed as the parent. Depending on which one is chosen, the middle child of the 3-node is either a left child or a right child in the red-black tree, and the slant of the line connecting parent and child is either left or right. These are sometimes called *left-leaning* and *right-leaning red-black trees*.

Both arrangements are valid; however, they may not contribute equally to balancing the tree. Let's look at the situation in a slightly larger context.

The top panel in [Figure 10-35](#) shows a 2-3-4 tree. The bottom panels show the left-leaning and right-leaning options for equivalent red-black trees derived from the transformation rules. The difference between them is the choice of which of the two data items in the 3-node at the top to make the parent. In the left-leaning option, node 80 becomes the parent, and in the right-leaning one, node 70 is chosen.

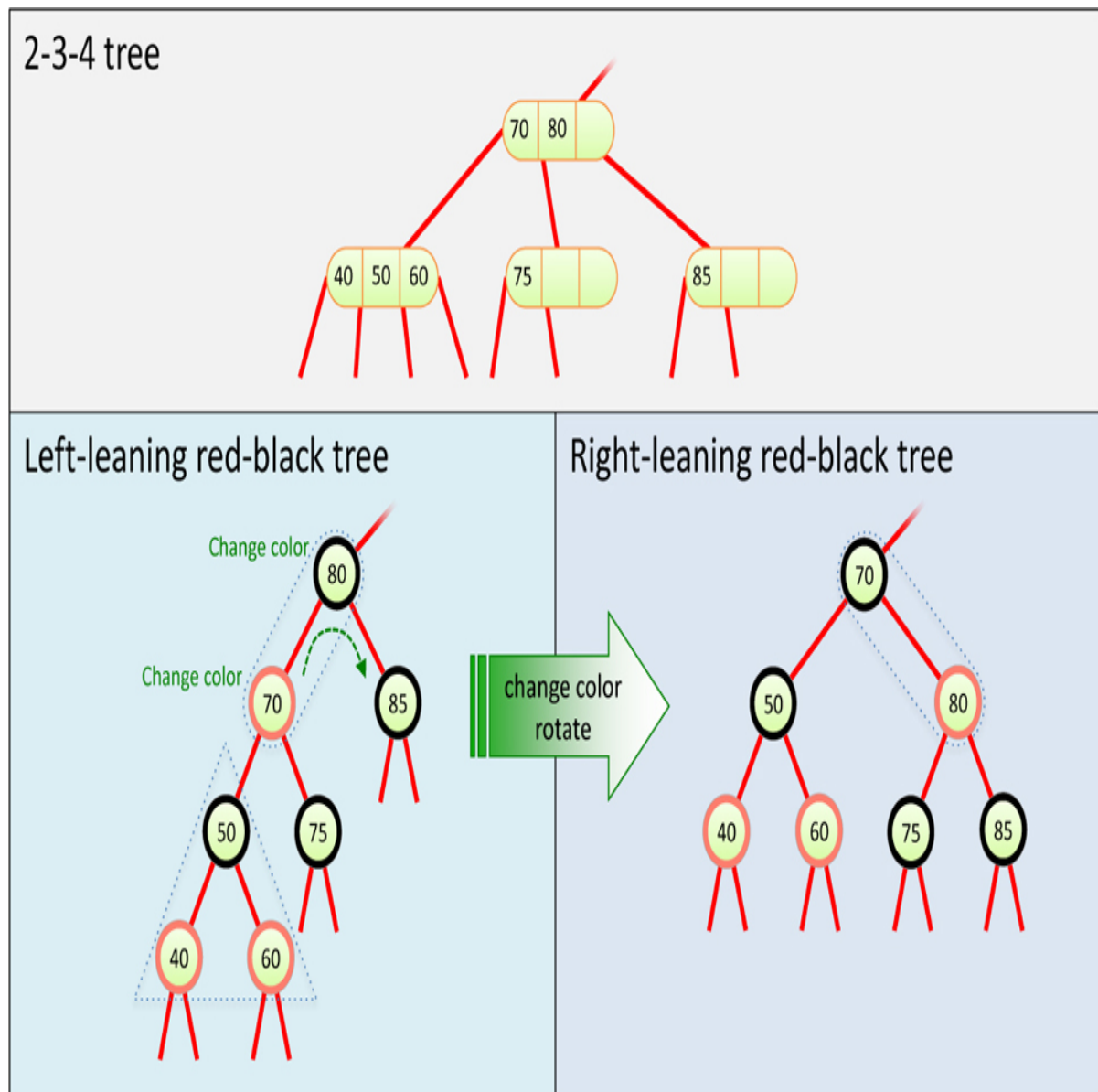


Figure 10-35 3-node transformation and equivalent rotation

Although these arrangements are equally valid, you can see that the tree on the left looks less balanced than the one on the right, even though both follow all the red-black rules. If you tried to insert an item with a key of 45 in the tree on the left, the insertion algorithm would descend the left branch and find that node 50 has two red children and perform a color swap. That would make node 50 red and violate Rule 3 with its parent, node 70. To fix that, you would apply the rule for Outside Grandchild conflicts. Amazingly, this rotation (performed on the tree without conflicts) results in the exact same tree as the right-leaning option!

Thus, you can see equivalence between rotations in red-black trees and the choice of which node to make the parent when transforming 2-3-4 trees to red-black trees. Although we don't show it, a similar equivalence can be seen for the double rotation necessary for inside grandchildren.

Red-Black Tree Implementation

If you're writing an insertion routine for red-black trees, all you would need to do (irony intended) is to write code to carry out the operations described in the preceding sections. As we noted, showing and describing such code are beyond the scope of this book. Nevertheless, here's what you would need to think about.

First, you'd need to add a red-black field to the `__Node` class. The constructor would create red nodes for insertion, by default.

You could adapt the insertion routine from the `BinarySearchTree` in [Chapter 8](#), the `Tree234` class in [Chapter 9](#), or the `AVLtree` of this chapter. On the way down to the insertion point, check if the current node is black and its two children are both red. If so, change the color of all three (unless the parent is the root, which must be kept black).

After a color swap, check that there are no violations of Rule 3. If so, perform the appropriate rotations: one for an outside grandchild, two for an inside grandchild.

When you reach a leaf node, insert the new node, making sure the node is colored red. Check again for red-red conflicts, and perform any necessary rotations.

Perhaps surprisingly, your software need not keep track of the height or black height of different parts of the tree (although you might want to check this during debugging). You need to check only for violations of Rule 3, a red parent with a red child, which can be done locally (unlike checks of black heights, Rule 4, which would require more complex bookkeeping like you did for the height in AVL trees).

If you perform the color swaps, color changes, and rotations described earlier, the black heights of the nodes take care of themselves, and the tree should remain balanced. The `RedBlackTree` Visualization tool reports black-height errors only to help you understand the rules; they don't need to be tracked by

the algorithm. Although the concepts of the red-black tree are a bit harder to grasp, the implementation can be faster than that of an AVL tree, even though both remain $O(\log N)$.

Summary

- Keeping a binary search tree balanced ensures that the time to find a given node is as short as possible.
- Inserting data that has already been sorted can create a maximally unbalanced tree, which will have search times of $O(N)$.
- You can measure tree balance by counting nodes or the length of paths in the left and right subtrees.
- You should measure balance at every node (every subtree), not just the root of the tree.
- In the AVL tree, each node keeps a field measuring its height.
- The height of a node is the number of nodes on the path to the deepest leaf node below it, including itself.
- The height of an empty subtree of a node is considered zero.
- A binary tree is fully balanced when the height difference of the left and right child of every node is at most one.
- AVL trees perform bottom-up correction of imbalances.
- Left and right rotations alter the positions and heights of subtrees.
- Rotations are implemented by changing the links between nodes of the tree.
- Rotations preserve the order of keys in the tree by moving the crossover subtree to the left or right of the top of the rotation.
- When inserting a node causes an AVL subtree to become left or right heavy, it makes one rotation for an outside grandchild and two rotations for an inside grandchild to correct the imbalance.

- Deleting a node from an AVL subtree involves finding the node to delete, finding the successor of the node to delete, moving the successor item up to the node to delete, deleting the successor node, and correcting any imbalance created by the deletion of the successor node.
- Imbalances caused by deletion can also be corrected using one or two rotations by checking the height difference at a node and the child where the deletion occurred.
- Each AVL node's height must be updated after rotations, deletions, and insertions are performed on its subtrees.
- AVL trees take $O(\log N)$ for search, insertion, and deletion operations. Traversal is $O(N)$.
- AVL trees consume $O(N)$ memory.
- In the red-black balancing scheme, each node is assigned a characteristic—a color that is either red or black.
- A set of four rules, called the red-black rules, specifies permissible ways that nodes of different colors can be arranged.
- The red-black rules are maintained while inserting and deleting each node.
- A color swap changes a black node with two red children to a red node with two black children.
- Rotations alter the heights of subtrees and sometimes cause violations of the red-black rules.
- Color swaps, and sometimes rotations with color flips, are applied while searching down the tree to find where a new node should be inserted. These swaps help return the tree to red-black correctness following an insertion.
- After a new node is inserted or an internal node undergoes a color swap, red-red conflicts are checked again. If a violation is found, appropriate rotations are carried out to make the tree red-black correct.

- These top-down adjustments result in the tree being balanced, or at least almost balanced.
- Red-black trees have the same big O performance as AVL trees do: $O(\log N)$ for search, insertion, and deletion operations. Traversal is $O(N)$.
- In the worst case, the search path in a red-black tree is twice the length of that of an AVL tree holding the same items.
- Red-black trees consume $O(N)$ memory.
- Red-black trees consume less memory per node than AVL trees because they need only one bit of storage for the node color, whereas AVL trees use an integer to track the height of the tree.
- Adding balancing to a binary tree has only a small negative effect on average performance and avoids worst-case performance when the data is already sorted.
- A red-black tree can be derived from any 2-3-4 tree, maintaining balance.
- Insertion and deletion operations on 2-3-4 trees can be shown to be equivalent to the corresponding operations in red-black trees.

Questions

These questions are intended as a self-test for readers. Answers may be found in [Appendix C](#).

1. A balanced binary search tree is desirable because it avoids slow performance when data is inserted _____.
2. In a balanced tree, there are roughly the same number of nodes in the left and right subtrees, and
 - a. the tree may need to be restructured during searches.
 - b. the paths from the root to all the leaf nodes are about the same length.
 - c. all left subtrees are the same height as all right subtrees.
 - d. the starting level of all subtrees is closely controlled.

3. Which of the following metrics guarantees a balanced binary tree if it is less than or equal to one?
 - a. the sum of the differences between the node count of the left and right subtrees, at every node of the tree
 - b. the difference in the number of nodes in the left and right subtrees of the root node
 - c. the absolute difference between the heights of the left and right subtrees, at every node of the tree
 - d. the height of the left subtree minus the height of the right subtree of the root node
4. AVL trees
 - a. track the number of nodes in the subtree below each node.
 - b. use average values of the keys in a subtree to speed up the search time.
 - c. insert each item with height 1 and then adjust that height after modifications to its subtrees in later operations.
 - d. use the difference in height of a node's subtrees to determine which subtree will store the next item inserted.
5. If an empty AVL tree has 100 items inserted into it, and every insertion causes either an even balance or a temporary left-heavy tree at the root, you can conclude that
 - a. the number of right rotations will equal or exceed the number of left rotations.
 - b. the tree will be partially unbalanced leading the average number of comparisons for a search to be around 50.
 - c. the last 50 items inserted will end up in the right subtree of the root.
 - d. only right rotations will be needed to correct imbalances.
6. Deleting an item from an AVL tree
 - a. is easiest when the node containing the item has two subtrees.
 - b. uses either one or two rotations to rebalance subtrees based on the height balance of (1) the subtree and (2) the child with the larger

height,

- c. uses fusion operations like 2-3-4 trees do in some situations.
 - d. uses the mirror image operations of inserting an item.
7. True or False: The red-black rules are a sequence of steps that rearrange the nodes in a tree to balance it.
8. A null child is
- a. a child that doesn't exist but will be created next as a placeholder to balance the tree.
 - b. a child with no children of its own.
 - c. a child whose key was deleted but was left in the tree as a soft-deletion.
 - d. a nonexistent left child of a node with a right child (or vice versa).
9. Which of the following is **not** a red-black rule?
- a. Every path from a root to a leaf, or to a null child, must contain the same number of black nodes.
 - b. If a node is black, its children must be red.
 - c. The root node is always black.
 - d. Every node must be either black or red.
10. The two possible actions used to balance a red-black tree are _____ and _____.
11. Newly inserted nodes are always colored _____.
12. A crossover node or subtree starts as a _____ and becomes a _____, or vice versa.
13. Which of the following is **not** true of red-black trees? Rotations may need to be made
- a. before a node is inserted.
 - b. after a node is inserted.
 - c. during a search for the insertion point.
 - d. when searching for a node with a given key.

14. A color swap involves changing the color of _____ and _____.
15. An outside grandchild is
- on the opposite side of its parent than its parent is of its sibling.
 - on the same side of its parent than its parent is of its parent.
 - one which is the left descendant of a right descendant (or vice versa).
 - on the opposite side of its parent than its sibling is of their grandparents.
16. True or False: When one rotation immediately follows another as part of single insertion at the lowest level of a red-black tree, they are in opposite directions.
17. Two rotations are necessary when
- the node is an inside grandchild and the parent is red.
 - the node is an inside grandchild and the parent is black.
 - the node is an outside grandchild and the parent is red.
 - the node is an outside grandchild and the parent is black.
18. Two color swaps are necessary when
- encountering a black node with two red children on deletion.
 - encountering a red node with two black children on insertion.
 - encountering a black node with two red children on insertion.
 - none of these situations.
19. In comparing the efficiency of AVL trees and red-black trees
- all operations in AVL trees are somewhat more efficient than those of red-black trees because they operate bottom-up.
 - AVL trees use more memory per node and have slower search than red-black trees.
 - red-black trees use fewer rotations to balance trees making them faster at insertion, deletion, and search.
 - both are $O(\log N)$ but the red-black rules can result in balanced trees with path lengths up to twice as long as those in an AVL tree, leading to slower search times in red-black trees.

20. The 2-3-4 trees ([Chapter 9](#))

- a. can be collapsed into 2-3 trees ([Chapter 9](#)), and in some cases, into a balanced binary trees ([Chapter 10](#)).
- b. can be transformed into AVL trees, but their balance must be adjusted through additional rotations.
- c. can be restructured using rotations into a red-black tree, although the balance of the red-black tree may be left-heavy or right-heavy depending on how 3-nodes are handled.
- d. can be mapped into equivalent red-black trees and the 4-node split operation has the same effect as a red-black color swap.

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

10-A Tree structures have been used by secret organizations such as resistance movements for a long time. The tree structure represents the hierarchy of the group. Group members know their leader—their parent node—and their direct subordinates—their child nodes. Knowing more than a few of their members, however, can be dangerous to the organization. An enemy who discovers one member may be able to discover the other parts of the group known to that member.

Which tree structure that you've studied ([Chapters 9 and 10](#)) would be best as the basis for organizing such a group? How would balancing the tree help or hurt the organization? If it helps, which method(s) of balancing that you've seen would be best in terms of preventing an enemy from knowing the size and structure of the organization if they captured one member?

10-B If an AVL tree uses a single byte to represent the height at each node, the height can range from 0 to 255. A tree with heights deeper than that would cause problems. How many items (nodes) could such an AVL tree hold without running in to that problem?

10-C If you haven't already, perform all of the experiments in the sections: "[Experiment 1: Inserting Two Red Nodes](#)," "[Experiment 2: Rotations](#),"

[“Experiment 3: Color Swaps,”](#) and [“Experiment 4: An Unbalanced Tree.”](#)

- 10-D Use the RedBlackTree Visualization tool and fill an empty tree with 31 random nodes. Repeat this 50 times. How often does the resulting tree satisfy all the red-black rules? How often is the tree grossly unbalanced (that is, have some node with a height difference of 3 or more)? How does this compare with randomly constructing similar trees in the Binary Search Tree Visualization tool?
- 10-E Do enough insertions to convince yourself that if red-black rules 1, 2, and 3 are followed exactly along with the color swaps and rotations described in this chapter, Rule 4 will take care of itself.

Programming Projects

Writing programs to solve the Programming Projects helps to solidify your understanding of the material and demonstrates how the chapter’s concepts are applied. (As noted in the Introduction, qualified instructors may obtain completed solutions to the Programming Projects on the publisher’s website.)

- 10.1 Write a method `isBalanced()` for the `AVLtree` class that verifies the entire tree is balanced. It should return `True` when the height difference of every node is `-1`, `0`, or `+1`, and `False` otherwise. Show that the tree starts balanced when it’s empty and remains balanced as nodes are inserted and later deleted.

For extra thoroughness, use Python’s `itertools` package to find every permutation of the order of six distinct keys to insert into the empty tree. Count how many of those insertion permutations produce balanced trees.

- 10.2 Write a method `howManyWithin()` for the `AVLtree` class that counts the number of items with keys within a low and a high value. Because the AVL tree is a binary search tree, not all nodes need to be investigated. For example, when visiting a node whose key is 20 when the low value of the range is 40, there’s no need to visit its left child branch because it cannot possibly have any keys in the range. The method should count keys that match the low or high value, if the high value is equal to or greater than the low (a non-empty range). Test your method on a variety

of ranges including empty ranges and ones that match exactly one key in the tree.

- 10.3 Perhaps you have used sets in math. They are used to group things like the set of all prime numbers or the set of all multiples of 3. Another similar concept is a *multiset*. In the standard set, an element is either in or out of the set and repeat instances of the element cannot be in the set. The multiset allows multiple instances of the same element in a set. The multiset tracks the count of each of its elements; if the count becomes zero, the element is no longer in the set. This same idea has been called *bag*, *aggregate*, *weighted set*, and other names in various places. It could be used, for example, to represent the coins in a bag or the cards in a player's hand in a game that has multiple copies of the same card. Python has a built-in `set` data type but not a specific multiset.

Use the `AVLtree` to implement a `Multiset` class and perform the basic set operations. The keys of the AVL tree are the elements of the multiset and their corresponding values store the count of that element. This allows for multisets of strings, integers, tuples, lists, and any other data type that can be ordered.

Your implementation of `Multiset` should include the following methods:

- `__len__()`—Returns the number of keys (distinct elements) stored in the multiset and allows the Python `len()` function to be used on multisets.
- `cardinality()`—Returns the total count of all elements stored in the multiset.
- `__str__()`—Returns a string representation of the multiset showing the keys and their counts, for example “[(A: 4) , (P: 1)]”. This allows the multiset to be printed or passed to the Python `str()` function.
- `__contains__(key)` —Returns `True` if the given key is in the multiset and `False` otherwise, allowing the multiset to be used in an expression such as `key in multiset`.
- `count(key)` —Returns the count of the given key in the multiset or zero if the key is not in the multiset.

- `add(key, count=1)` —Add a key to the multiset with an optional `count` parameter that defaults to 1, allowing multiple copies to be added in one call. Another way to describe this method is that it increases the count for the given key by the `count` parameter.
- `remove(key, count=1)` —Remove a certain number of copies of key from the multiset. The number of copies is an optional parameter that defaults to 1. If the number of copies being removed is equal to or higher than the current count for the key, the key should be removed from the underlying AVL tree.

Show your implementation working on an empty multiset, adding some items, showing the string representing the multiset, showing the length and cardinality for the whole multiset, showing the count for individual keys, then deleting some elements, and then showing more counts after the deletions.

10.4 Extend the `Multiset` implementation from Programming Project 10.3 to add the `union()` and `intersection()` methods. The union of two multisets is like the union of a regular set except that the count for an element in the union is the maximum of its count in the input multisets. The intersection of two multisets has counts that are the minimum of the counts for the element in the input multisets. For both operations, elements that are not in a multiset have a count of zero.

The `union()` method of `Multiset` should take another multiset as input and return a new multiset containing the union. It's more difficult to write it as a method that modifies one of the input multisets because you must traverse the underlying AVL tree while modifying it. Similarly, the `intersection()` method should take another multiset as input and returns a new multiset containing the intersection.

To perform the union and intersection operations, you need to traverse the items in the underlying AVL trees. For this, you can look at the `traverse()` generator used for 2-3-4 trees in [Chapter 9](#) or the code provided for the AVL tree with this textbook. To perform the operations efficiently, you can generate iterators for both multisets and visit all the elements in order. This operation is somewhat like mergesort, which takes two sorted sequences and produces a new combined sequence in sorted order. At each step, the algorithm looks at the element with the

lowest key in the two sequences and determines what should go in the output multiset.

Show your implementation of union and intersection by adding a variety of elements to two multisets, ensuring some elements are in both sets while others are only in one but not the other. Also ensure that the counts of some of the common elements are different in the two multisets. Show the union and intersection multisets formed by combining those two along with their length and cardinality.

- 10.5 Measure the performance of the AVL tree by altering its structure to store and report statistics on the number of operations performed. In the constructor for an `AVLtree`, create storage for the number of calls to `updateheight()`, `heightdiff()`, `rotateRight()`, and `rotateLeft()` that are all initially zero. Add methods to get the values for these statistics and to clear them back to zero. Write a `__len__()` method that gets the count of the number of items in the `AVLtree` and a `height()` method that gets the height of the root node.

Use the new methods to get the absolute counts for each of the statistics as you insert 100 or more items and then delete them. Show the counts for insertions and deletions and the counts divided by the log of the size of the tree; that is, divide by `math.log(len(tree))`, when the size is four or above to always have a positive denominator and at least two levels in the tree. It's helpful to print the statistics in fixed width columns (see Python's `format` function for strings) so they look something like:

N	H	updHgt	hgtDif	RotLft	RotRgt	updHgt	hgtDif	RotLft	RotRgt
84	7	6	5	0	1	1.354	1.128	0.000	0.226

The first row shows the abbreviated titles of the statistics. It shows the number of nodes, `N`; the height of the tree, `H`; the four raw statistics; and the four ratios of those counts to the log `N` value. Print out about 15 samples of these statistics.

If the efficiency of the operations is $O(\log N)$, then the ratios should be bounded and relatively stable. They will be largest when there are the most rotations needed to rebalance the tree. Keep track of the maximum ratios and print them after all the items have been inserted and then

again after all have been deleted. The ratios should remain bounded even if you increase the number of items to a thousand.

In fact, the worst ratios occur when the tree is small because the denominator is the smallest. For example, look at the subtree rooted at node 72 in [Figure 10-5](#). If you forget about the rest of the tree and assume node 72 is the root, then it is a balanced AVL tree because all 12 of its nodes have a height difference of one or less. If you then delete node 69 from that tree, the left side becomes unbalanced. It requires two rotations to balance the left subtree because the inside grandchild must be promoted. The root is now unbalanced with a left side of height 2 and a right side of height 4. That requires a left rotation to fix, and it requires a further right rotation because the right subtree is left heavy. That's four rotations total, which each cause two calls to `updateHeight()`. There are two more calls to `updateHeight()` from the descent down to the node to delete, node 69, which leaves a total of 10 calls. The number of calls divided by the log of the number of nodes, 12 (before the deletion), is a little more than 4.0 when using the `math.log()` function, which is what's called the natural logarithm. Your results should show ratios less than 4 for larger trees.

11. Hash Tables

In This Chapter

- [Introduction to Hashing](#)
- [Open Addressing](#)
- [Separate Chaining](#)
- [Hash Functions](#)
- [Hashing Efficiency](#)
- [Hashing and External Storage](#)

A **hash table** is a data structure that offers very fast insertion and searching. When you first hear about them, hash tables sound almost too good to be true. No matter how many data items there are, insertion and searching (and sometimes deletion) can take close to constant time: $O(1)$ in Big O notation. In practice this is just a few machine instructions.

For a human user of a hash table, this amount of time is essentially instantaneous. It's so fast that computer programs typically use hash tables when they need to look up hundreds of thousands of items in less than a second (as in spell checking or in auto-completion). Hash tables are significantly faster than trees, which, as you learned in the preceding chapters, operate in relatively fast $O(\log N)$ time. Not only are they fast, hash tables are relatively easy to program.

Despite these amazing features, hash tables have several disadvantages. They're based on arrays, and expanding arrays after they've been allocated can cause challenges. If there will be many deletions after inserting many items, there can be significant amounts of unused memory. For some kinds of hash tables, performance may degrade catastrophically when a table becomes too full, so programmers need to have a fairly accurate idea of how many data

items will be stored (or be prepared to periodically transfer data to a larger hash table, a time-consuming process).

Also, there's no convenient way to visit the items in a hash table in any kind of order (such as from smallest to largest). If you need this kind of traversal, you'll need to look elsewhere.

However, if you don't need to visit items in order, and you can predict in advance the size of your database or accept some extra memory usage and a tiny bit of slowness as the database is built up, hash tables are unparalleled in speed and convenience.

Introduction to Hashing

In this section we introduce hash tables and hashing. The most important concept is how a range of key values is transformed into a range of array index values. In a hash table, this transformation is accomplished with a **hash function**. For certain kinds of keys, however, no hash function is necessary; the key values can be used directly as array indices. Let's look at this simpler situation first and then go on to look at how hash functions can be used when keys aren't distributed in such an orderly fashion.

Bank Account Numbers as Keys

Suppose you're writing a program to access the bank accounts of a small bank. Let's say the bank is fairly new and has only 10,000 accounts. Each account record requires 1,000 bytes of storage. Thus, you can store the entire database in only 10 megabytes, which will easily fit in your computer's memory.

The bank director has specified that she wants the fastest possible access to any individual record. Also, every account has been given a number from 0 (for the first account created) to 9,999 (for the most recently created one). These account numbers can be used as keys to access the records; in fact, access by other keys is deemed unnecessary. Accounts are seldom closed, but even when they are, their records remain in the database for reference (to answer questions about past activity). What sort of data structure should you use in this situation?

Index Numbers as Keys

One possibility is a simple array. Each account record occupies one cell of the array, and the index number of the cell is the account number for that record. This type of array is shown in [Figure 11-1](#).

nAccounts = 10,000

Account number = array index	0	1	2		9,998	9,999		15,000
Surname	Tanaguchi	Samuels	Sharma		Garibaldi	Samuelson		<empty>
Givenname	Reiko	Anna Marie	Ajay		Arturo	Pia		
Account type	checking	savings	savings		certificate	savings		
Date Open	1990-03-21	1990-06-04	1991-11-23		2019-04-30	2019-05-01		
Date Closed			2010-12-04					
Branch code	1	1	1		17	5		
Interest rate	0.05	0.15	0.12		2.15	0.45		
Balance	245.32	1,845.36	0.00		10,495.36	8,945.65		

Record fields

Array

Figure 11-1 *Account numbers as array indices*

As you know, accessing a specified array element is very fast if you know its index number. The clerk looking up what account a check is drawn from knows that it comes from, say, number 72, so he enters that number, and the program goes instantly to index number 72 in the array. A single program statement is all that's necessary:

```
accountRec = databaseArray[72]
```

Adding a new account is also very quick: you insert it just past the last occupied element. If there are currently 9,300 accounts, the next new record would go in cell 9,300. Again, a single statement inserts the new record:

```
databaseArray[nAccounts] = newAccountRecord()
```

The count of the number of accounts would be incremented like this:

```
nAccounts += 1
```

Presumably, the array is made somewhat larger than the current number of accounts, to allow room for expansion, but not much expansion is anticipated, or at least it needs to be done only infrequently, such as once a month.

Not Always So Orderly

The speed and simplicity of data access using this array-based database make it very attractive. This example, however, works only because the keys are unusually well organized. They run sequentially from 0 to a known maximum, and this maximum is a reasonable size for an array. There are no deletions, so memory-wasting gaps don't develop in the sequence. New items can be added sequentially at the end of the array, and the array doesn't need to be much larger than the current number of items.

A Dictionary

In many situations the keys are not so well behaved, as in the bank account database just described. The classic example is a dictionary. If you want to put every word of an English-language dictionary, from *a* to *zyzzzyva* (yes, it's a word), into your computer's memory so they can be accessed quickly, a hash table is a good choice.

A similar widely used application for hash tables is in computer-language compilers, which maintain a **symbol table** in a hash table (although balanced binary trees are sometimes used). The symbol table holds all the variable and function names made up by the programmers, along with the address (or register) where they can be found in memory. The program needs to access these names very quickly, so a hash table is the preferred data structure.

Coming back to natural languages, let's say you want to store a 50,000-word English-language dictionary in main memory. You would like every word to occupy its own cell in a 50,000-cell array, so you can access the word's record (with definitions, parts of speech, etymology, and so on) using an index number. This approach makes access very fast, but what's the relationship of these index numbers to the words? Given the word *ambiguous*, for example, how do you find its index number?